

Python and AI on Lovelace

Vincent Drach

June 2026

School of Engineering Computing and Mathematics
vincent.drach@plymouth.ac.uk

Today's goal and outline

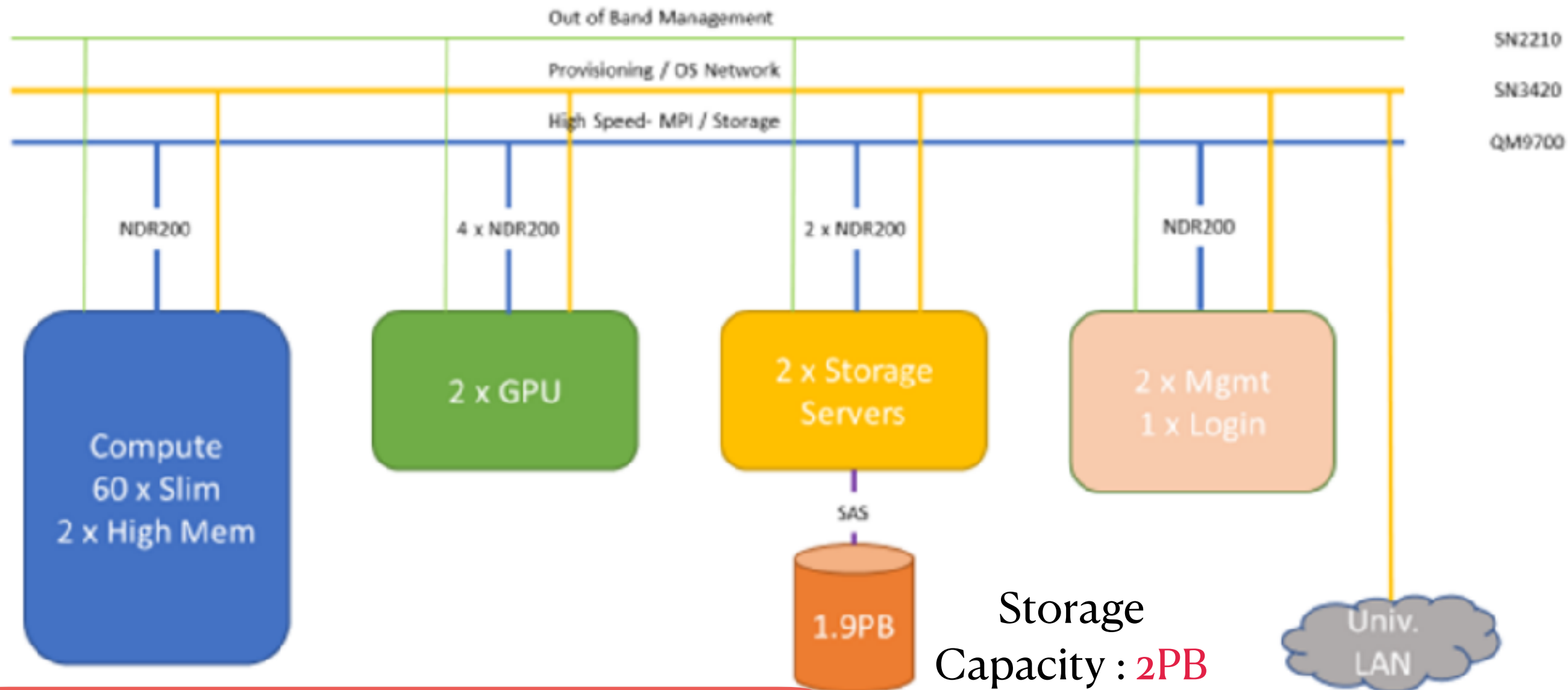
- **Goal** : How to run efficiently an AI workflow on Lovelace.

Outline:

- Connecting to *Lovelace*
- *A classification problem : ECG dataset*
- Preprocessing
- GPUs
- Training profiling and optimisations
- Parallelisation strategies and scaling
- (Advanced preprocessing)
- Key takeaways : AI workflow and GPUs

Introduction and connecting to the HPC facility

Lovelace system (March 2025)



60x CPU nodes :

AMD EPYC 64 cores per nodes
384 GB of RAM

2x High Mem nodes :

AMD EPYC 64 cores per nodes
1,536GB of RAM

GPU Server #1

8 x L40S Nvidia GPUs
1,536GB of RAM

GPU Server #2

4 x H100 SXM5 80G Nvidia GPUs
Fully meshed NVlink
1,536GB of RAM

NVIDIA QM9700 NDR Managed
Infiniband Switch.

400Gbit/s

Computing Nodes

Login in Lovelace (Linux)

- On University machine: download mobaxterm
- Alternative for windows users : WSL (Window Subsystem for Linux)

See Useful Link section on <https://docs.lovelace.plymouth.ac.uk/>

- Start MobaXterm
- Type :

ssh username@lovelace.plymouth.ac.uk

- Enter your password
- You are now connected on the head node of Lovelace.

Common features of HPC machines

- A Linux Machine that can be access remotely
- **Module Manager:** A way to load **your** tools
- **Job scheduler:** that dispatches and orchestrates jobs on the computing nodes - your jobs might not run immediately
- You can/must **fine-tune your environment** to your need

Navigating the terminal

First step with the terminal

- *The shell*: a command line interpreter to interact with the kernel. It can be started using the “*terminal*” application. There are many of them the default is called “*bash*” for “Bourne-Again shell”.
- Some commands:
 - ★ **pwd**: Print working directory will tell you what directory you are in.
 - ★ **cd** *path*– Change Directory
 - ★ **ls** – Lists files & directories
 - ★ **w** – display who is logged in and what they are doing
 - ★ **rm** *filename* – Remove the file named filename
 - ★ **mkdir** *dirname* - make directories

Linux filesystem hierarchy

- **Linux uses a hierarchical file system:** everything starts from the root directory `/`, and all files, directories, devices, and storage locations are organized beneath it like branches of a tree..
 - ★ **/**: the root of the entire filesystem
 - ★ **/users** : *location of the “home folder”*
 - ★ **/scratch/** : *for storage*

Note : Nothing is backed up! You are responsible for your data.

Example

- Example type

ls -l

(-l is an option which stands for long listing format)

- You can see files in black, directory in blue, the date of last modification among other things.

```
vincent@MATH2607:~$ ls -l
total 76
drwx--xr-x 2 vincent vincent 4096 Nov 11 2016 C
-rw-rw-r-- 1 vincent vincent 593 Sep 22 09:47 cluster_config.sh
drwx--xr-x 2 vincent vincent 4096 Oct 21 2016 Desktop
drwx--xr-x 2 vincent vincent 4096 Oct 21 2016 Documents
drwx--xr-x 2 vincent vincent 4096 Oct 21 2016 Downloads
-rw----r-- 1 vincent vincent 8980 Oct 20 2016 examples.desktop
drwx--xr-x 2 vincent vincent 4096 Jan 26 2017 lecture1
-rw-rw-r-- 1 vincent vincent 553 Sep 20 17:21 lightinterface.sh
drwx--xr-x 2 vincent vincent 4096 Oct 21 2016 Music
-rw----r-- 1 vincent vincent 365 Oct 21 2016 notes_python
drwxrwxr-x 2 vincent vincent 4096 Sep 22 11:08 old_stuff
drwx--xr-x 2 vincent vincent 4096 Oct 21 2016 Pictures
drwx--xr-x 2 vincent vincent 4096 Oct 21 2016 Public
drwx--xr-x 2 vincent vincent 4096 Oct 21 2016 Templates
drwx--x--x 4 vincent vincent 4096 Jan 16 2017 tmp
drwx--xr-x 2 vincent vincent 4096 Oct 21 2016 Videos
drwx--xr-x 2 vincent vincent 4096 Nov 1 2016 workshop
vincent@MATH2607:~$
```

Examples

- Current location: `pwd`
- Examples:

```
[vincent@MATH2607:~$ pwd
/home/vincent
[vincent@MATH2607:~$ ls
C  cluster_config.sh  Desktop  Documents  Downloads  examples.desktop  lecture1  lightinterface.sh  Music  notes_python
[vincent@MATH2607:~$ ls /home/vincent
C  cluster_config.sh  Desktop  Documents  Downloads  examples.desktop  lecture1  lightinterface.sh  Music  notes_python
[vincent@MATH2607:~$ ls /home
cchristopher  charlotte  cmcneile  harry  james  jason  jennifer  lost+found  nick  pati  rago  rhea  sam  sean  test1
[vincent@MATH2607:~$
```

- Note that `ls` return what is inside the directory where you are
- `ls /home` return what is in `/home`:
- A *path* is a means to get to a particular file or directory on the system.

More on paths

- Path can be **absolute** or **relative**:

```
[vincent@MATH2607:~$ pwd
/home/vincent
[vincent@MATH2607:~$ ls
C  cluster_config.sh  Desktop  Documents  Downloads  examples.desktop  lecture1  lightinterface.sh  Music
[vincent@MATH2607:~$ ls Documents/
mydoc.txt
```

← relative path

```
[vincent@MATH2607:~$
[vincent@MATH2607:~$ ls /home/vincent/Documents/
mydoc.txt
```

← Absolute path

```
vincent@MATH2607:~$
```

- Shortcuts:

- ★ `~` (tilde) - shortcut for your home directory: `ls ~`
- ★ `.` (dot) - This is a reference to the **current** directory. Try: `ls ./Documents`
- ★ `..` (dotdot)- This is a reference to the **parent** directory. Try: `ls ..`

Example

- Playing a bit with ls:

```
[vincent@MATH2607:~$ pwd
/home/vincent
[vincent@MATH2607:~$ ls
C                Desktop      Downloads      lecture1        Music           old_stuff      Public         tmp            workshop
cluster_config.sh Documents    examples.desktop lightinterface.sh notes_python    Pictures        Templates      Videos
[vincent@MATH2607:~$ ls .
C                Desktop      Downloads      lecture1        Music           old_stuff      Public         tmp            workshop
cluster_config.sh Documents    examples.desktop lightinterface.sh notes_python    Pictures        Templates      Videos
[vincent@MATH2607:~$ ls ../
cchristopher  cmcneile  james  jennifer  nick  rago  sam  test1  visitor003
charlotte     harry     jason  lost+found peti  rhea  sean  vincent zachary
[vincent@MATH2607:~$ ls ../../
bin  cdrom  etc  initrd.img  lib  lost+found  mnt  proc  run  snap  sys  usr  vmlinuz
boot dev  home  initrd.img.old lib64 media  opt  root  sbin  srv  tmp  var  vmlinuz.old
[vincent@MATH2607:~$ ls /
bin  cdrom  etc  initrd.img  lib  lost+found  mnt  proc  run  snap  sys  usr  vmlinuz
boot dev  home  initrd.img.old lib64 media  opt  root  sbin  srv  tmp  var  vmlinuz.old
vincent@MATH2607:~$ █
```

A few remarks

- Linux is an extensionless file system (you can name an executable file.txt if you want... But it would be misleading)
- Luckily there is a command to determining what type of file a particular file is:
 - ★ **file**-- obtain information about what type of file a file or directory is.
- The manual:
 - ★ **man <command>** - Look up the manual page for a particular command. (e.g try man ls) [press q to quit the manual, space or up and down arrows to scroll down]
 - ★ Type **/-l** in the ls manual to look for “-l”
 - ★ then **n** to jump to the next iteration
 - ★ Look at the ls option : -a,-l,-r,-t for instance

Creating directories & files

- To create a directory: try in your home folder: `mkdir testdir`
- To remove it : `rmdir testdir` (note that testdir must be empty for rmdir to work)
- To create an empty file called “testfile” you can type `touch testfile`
- To create a copy “testfile” in type `cp testfile testfile_duplicate`
- To move “testfile” to a directory type `mv testfile ./testdir/`
- To rename move “testfile_duplicate” to a new file called “testfile2” type `mv testfile_duplicate testfile2`
- Copy the directory `/tmp/HPC_training` to your home folder. (Use `cp -r`, the option `-r` stands for recursive, all subdirectories will also be copied)

Tab completion

- Typing path can be tedious...
- When you start typing a path you may hit the **Tab key** on your keyboard at any time which will invoke an **auto complete action**. If nothing happens then that means there are several possibilities. If you hit Tab again it will show you those possibilities.
- E.g

```
[vincent@MATH2607:/home$ pwd
/home
[vincent@MATH2607:/home$ ls
cchristopher  cmcneile  james  jennifer  nick  rago  sam  test1  visitor003
charlotte     harry     jason  lost+found  pati  rhea  sean  vincent  zachary
[vincent@MATH2607:/home$ ls
cchristopher/ harry/      jennifer/  pati/      sam/      vincent/
charlotte/    james/     lost+found/ rago/      sean/     visitor003/
cmcneile/     jason/     nick/      rhea/      test1/    zachary/
[vincent@MATH2607:/home$ ls
```

I typed twice tab

Creating directories & files

```
test1@comp4:~$ pwd
/home/test1
test1@comp4:~$ ls
C          Desktop  Downloads  lecture1  Music  notes_python  Public  tmp  workshop
cluster_config.sh Documents examples.desktop lightinterface.sh new2 Pictures Templates Videos
test1@comp4:~$ mkdir testdir
test1@comp4:~$ ls
C          Desktop  Downloads  lecture1  Music  notes_python  Public  testdir  Videos  workshop
cluster_config.sh Documents examples.desktop lightinterface.sh new2 Pictures Templates tmp      workshop
test1@comp4:~$ rmdir testdir
test1@comp4:~$ ls
C          Desktop  Downloads  lecture1  Music  notes_python  Public  tmp  workshop
cluster_config.sh Documents examples.desktop lightinterface.sh new2 Pictures Templates Videos
test1@comp4:~$ touch testfile
test1@comp4:~$ ls
C          Desktop  Downloads  lecture1  Music  notes_python  Public  testfile  Videos  workshop
cluster_config.sh Documents examples.desktop lightinterface.sh new2 Pictures Templates tmp      workshop
test1@comp4:~$ cp testfile ./Documents/
test1@comp4:~$ cp testfile new
test1@comp4:~$ ls
C          Documents  lecture1  new  Pictures  testfile  workshop
cluster_config.sh Downloads lightinterface.sh new2  Public  tmp
Desktop      examples.desktop Music      notes_python Templates Videos
test1@comp4:~$ ls Documents/
testfile
test1@comp4:~$ mv testfile ./Documents/
test1@comp4:~$ ls
C          Desktop  Downloads  lecture1  Music  new2  Pictures  Templates  Videos  workshop
cluster_config.sh Documents examples.desktop lightinterface.sh new  notes_python Public  tmp      workshop
test1@comp4:~$ ls Documents/
testfile
test1@comp4:~$ mv new new2
test1@comp4:~$ ls
C          Desktop  Downloads  lecture1  Music  notes_python  Public  tmp  workshop
cluster_config.sh Documents examples.desktop lightinterface.sh new2 Pictures Templates Videos
test1@comp4:~$ rm new2
test1@comp4:~$ ls
C          Desktop  Downloads  lecture1  Music  Pictures  Templates  Videos  workshop
cluster_config.sh Documents examples.desktop lightinterface.sh notes_python Public  tmp      workshop
test1@comp4:~$
```

Displaying the contents of a file

- Most basic commands :
 - ★ **cat** - reads files sequentially, writing them to [standard output](#)
 - ★ **more or less** - to view (but not modify) the contents of a [text file](#) on screen [q to quit, space to scroll down, b to scroll back]
- Try to display the content of :
 - ★ The file hostname in **/etc**
 - ★ The file cp in /bin
 - ★ The file syslog in /var/log
- Editors without graphical interface:
 - ★ nano: text editor using a command line interface
 - ★ Emacs
 - ★ ...

Combining commands

- The pipe command `|`: allows to send the output of one command to another

```
[(base) Vincents-MBP:linux vincent$ ls
log_ls          out_13_L1024_beta0.2 out_2_L1024_beta0.2 out_6_L1024_beta0.2
out_10_L1024_beta0.2 out_14_L1024_beta0.2 out_3_L1024_beta0.2 out_7_L1024_beta0.2
out_11_L1024_beta0.2 out_15_L1024_beta0.2 out_4_L1024_beta0.2 out_8_L1024_beta0.2
out_12_L1024_beta0.2 out_1_L1024_beta0.2 out_5_L1024_beta0.2 out_9_L1024_beta0.2
[(base) Vincents-MBP:linux vincent$ cat log_ls
out_10_L1024_beta0.2
out_11_L1024_beta0.2
out_12_L1024_beta0.2
out_13_L1024_beta0.2
out_14_L1024_beta0.2
out_15_L1024_beta0.2
out_1_L1024_beta0.2
out_2_L1024_beta0.2
out_3_L1024_beta0.2
out_4_L1024_beta0.2
out_5_L1024_beta0.2
out_6_L1024_beta0.2
out_7_L1024_beta0.2
out_8_L1024_beta0.2
out_9_L1024_beta0.2
[(base) Vincents-MBP:linux vincent$ cat log_ls | wc -l
15
```

Variables in bash

- A **bash variable** can contain a number, a character, a string of characters.
- No need to declare the variable. The assignment creates the variable.
- To display the value of a variable *VAR* use the **echo** command with the name of variable preceded by a **\$** symbol:, eg:

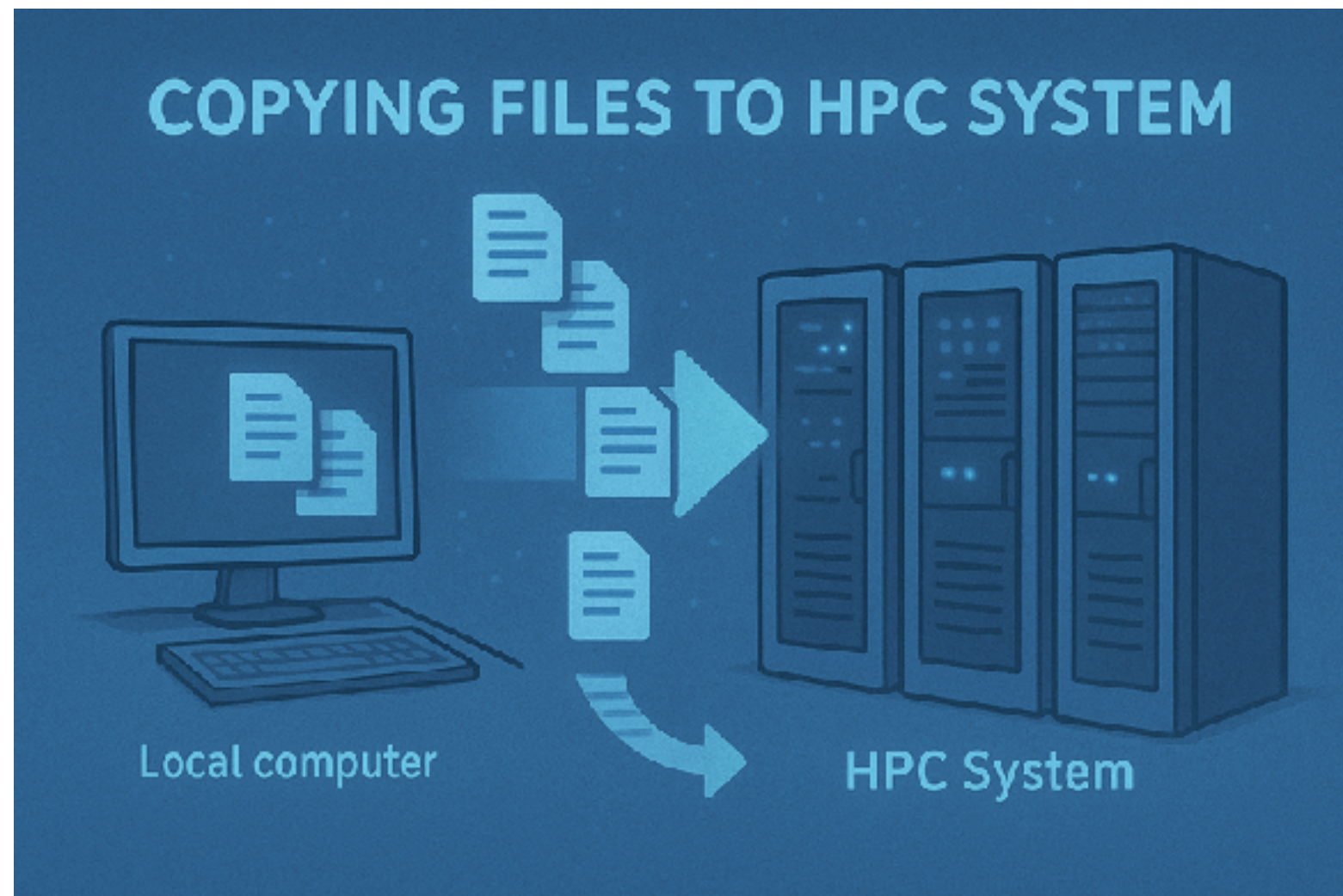
```
(base) Vincents-MBP:linux vincent$ VAR=something
(base) Vincents-MBP:linux vincent$ echo $VAR
something
(base) Vincents-MBP:linux vincent$
```

- Or
(base) Vincents-MBP:linux vincent\$ echo "the content of the variable VAR is \$VAR"
the content of the variable VAR is something
- In most cases : **\$var** and **\${var}** are the same (The braces are only needed to resolve ambiguity in expressions)
- To define a variable that contains the output of a command : **var=\$(date)**

Copy files back and forth

Copying files back and forth

- The login node is not your own machine. You will need to move data from your [local machine](#) to the login node and most likely move the output of your simulations out of the [remote machine](#) HPC system.
- Which machine is local or remote depends on which you are executing the commands!



Copy files over ssh

- Command to copy from/to a remote host: **scp** (secure copy)

- Syntax:

`scp Source Destination`

- Examples:

```
scp SourceFile user@lovelace.plymouth.ac.uk:./
```

```
scp SourceFile user@lovelace.plymouth.ac.uk:./directory/TargetFile .
```

To copy a directory to your home directory:

```
scp -r dir user@lovelace.plymouth.ac.uk:./
```

To copy from the remote host to your local machine in the present directory:

```
scp user@lovelace.plymouth.ac.uk:./directory/File .
```

- There are other tools to copy files over ssh. A useful one is **rsync**.

- Exercise:

- Create a file using notepad++ in the Document folder on your local (Windows) machine. Copy it on lovelace. Go on lovelace and visualise the content of the file.
- Create a file on lovelace called hello (for instance type: `touch hello`), copy the file back to your local Windows machine.

Modules

An HPC machine is flexible tool

- **We provide and maintain a flexible computing environment.**
- **Each user must fine tune the system to its need**
- The system admin team provides a number of pre-configured compilers/software/libraries, that can be loaded using a specific command.

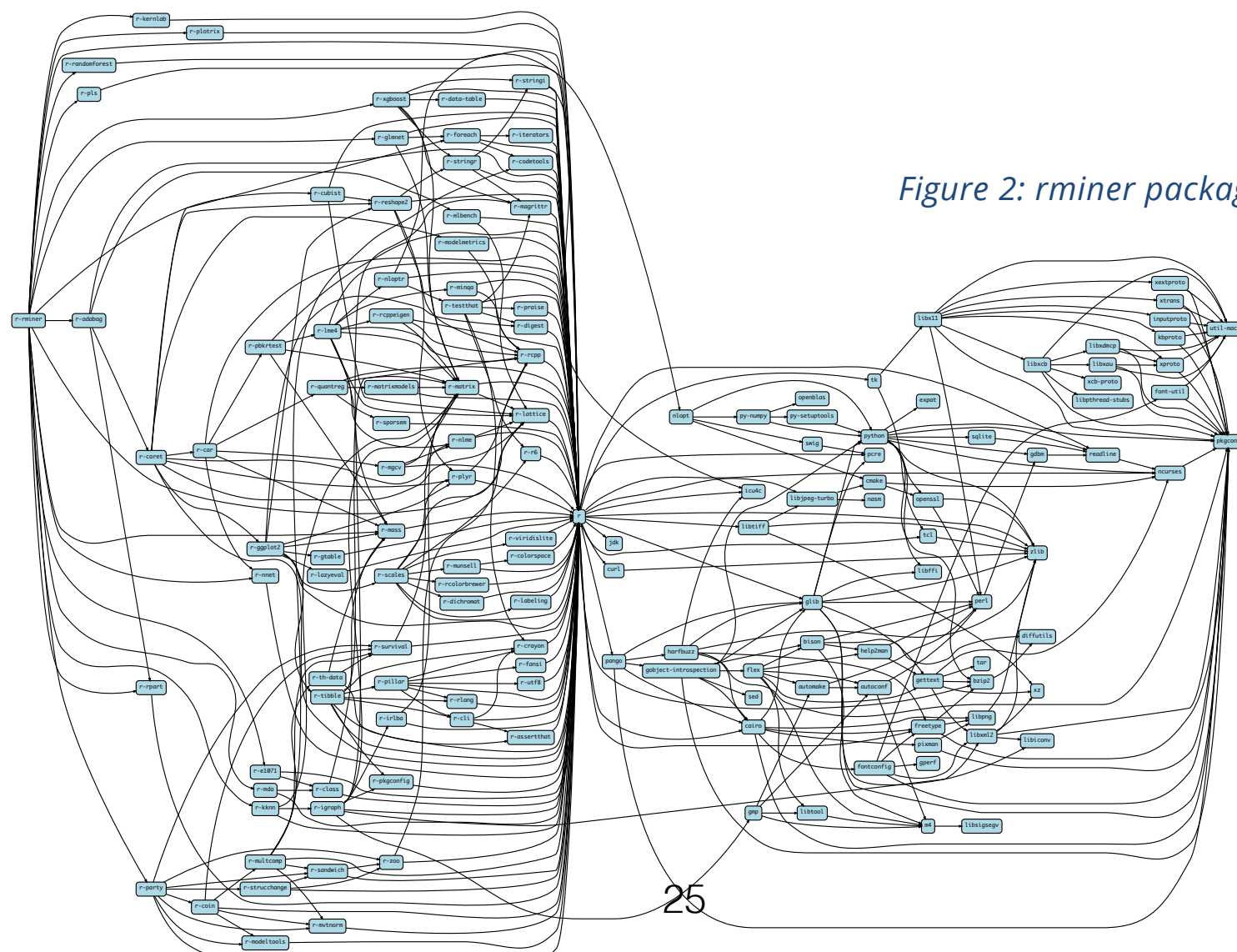


Figure 2: *rminer* package, with 150 dependencies.

Source: lInl (spack)

The module command

- To list all currently available module type:

`module avail`

- If you partially know the name of the module your are looking for type

`module spider py-tensorflow`

- To load a module type

`module load ModuleName`

The command will configure the shell for an application by modifying the environment variables

- To find help on a given module

`module help ModuleName`

The module command

- If you want to start from a fresh environment type:

`module purge`

- Many user will need a compiler, we currently advise to start by the following

`module load gcc`

- If you need MPI support, use:

`module load openmpi`

- Exercise load your favourite module.
- If your favourite software is missing, contact support.

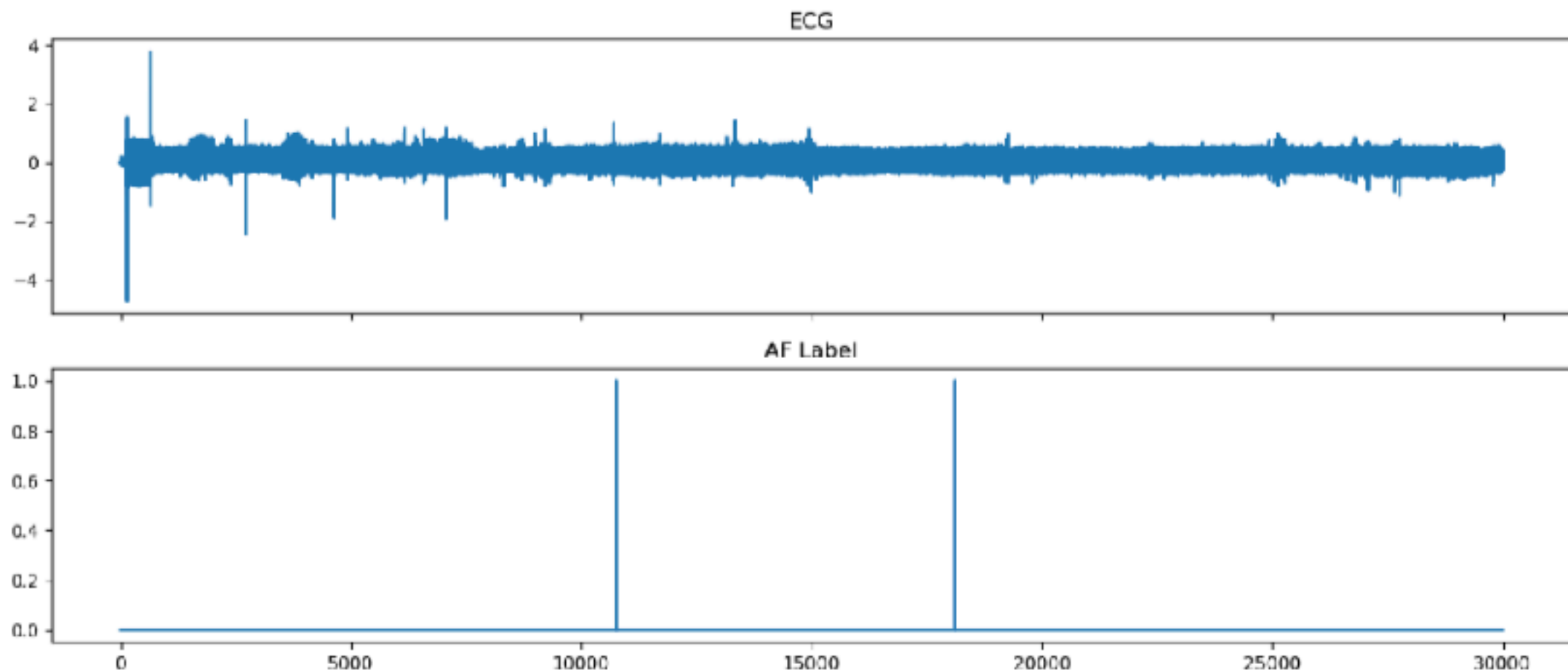
A classification problem

The Long-Term AF Database (LTAfDB)

- Data includes 84 long-term ECG recordings of subjects with a condition called sustained atrial fibrillation (AF).

Signal characteristics:

- 2 simultaneously recorded ECG leads
- Sampling rate: 128 Hz, 12-bit resolution over a 20 mV range
- Recording durations: typically 24–25 hours per subject
- Beat-by-beat annotations + rhythm-change annotations (AF episodes, terminations) manually verified



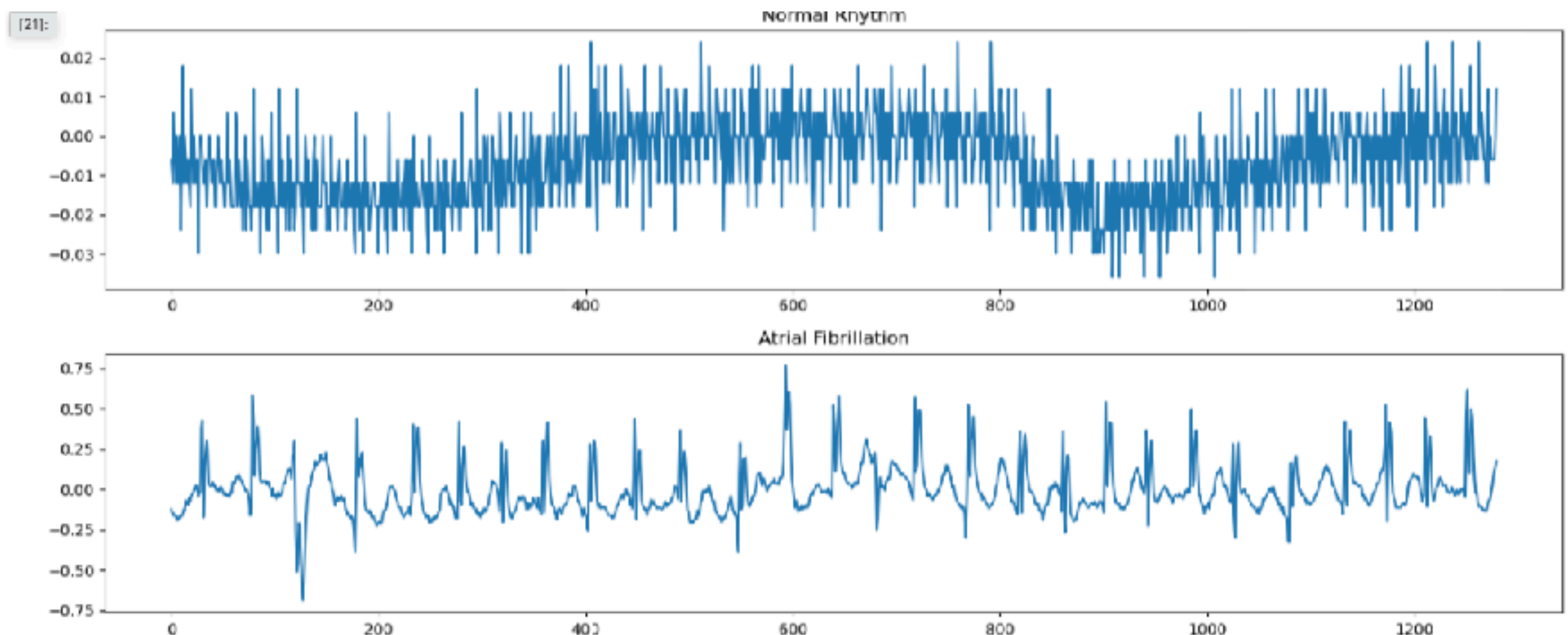
Classification problem

Task: binary classification per window — is this window during an AF episode, or not?

Windowing procedure:

- Each long recording is sliced into fixed-length 1280-sample windows = 10 seconds of ECG
- Each window inherits its label from the rhythm annotation covering it (AF vs non-AF)
- Across all 84 patients × 24h of recording: 705,732 labeled windows

Example:



Tools for today

- My repo is available in : https://github.com/vdrach/ecg_ai/tree/main#
- You can clone it by typing

```
git clone https://github.com/vdrach/ecg_ai.git
```
- A few comments:
 - The modules we will need today are listed in the file *modules_for_ecg_ai*
 - The file *HPC_pytorch_AI.toml* is an input file for a tool we provide [will be discussed on the website soon]
 - Some operations are automatised using a Makefile
 - For instance : `make data-small` [it takes for ever]
 - `make copy-data` [get the raw data from a central location]
 - On your own machine you could type `make preprocess` ; to preprocess the data. Makefiles are useful to automatise some tasks.

Environment configuration - linking with Jupyterhub

- Inside *HPC_pytorch_AI.toml*

```
name = "HPC_pytorch_AI"
human_name = "HPC Pytorch "
pip_packages = "wfdb fsspec nvidia-ml-py torch-tb-profiler "
modules = [
    "spack-modules/293",
    "compilers/llvm/14.0.6-xpvm6vv",
    "openmpi/5.0.5-pplxwdu",
    "python-science/py-numpy-py3.11/1.26.4-k5adanv",
    "python-science/py-numba-py3.11/0.59.1-ijufvg",
    "deeplearning/py-tensorboard-py3.11/2.17.1-c55bk65",
    "py-virtualenv-py3.11/20.26.6-xqjpfog",
    "deeplearning/py-torch-py3.11/2.8.0-3feiad",
    "python-science/py-matplotlib-py3.11/3.10.6-ogsaoll",
    "python-science/py-pandas-py3.11/2.3.2-wr7hjzs",
    "python-science/py-scipy-py3.11/1.16.1-g5b6aib",
]
```

Then run the following line in your terminal (login node). Of course make sure the toml is available.

```
/scratch/scripts/setup_jupyter HPC_pytorch_AI.toml
```

You can change name, human_name, pip_packages and modules to suit your needs. In this example, "HPC Pytorch" will then be one of the kernels available in jupyterhub.

Run it ! [it will take some time]

Accessing Lovelace through the web browser

- Look for an email by “hpc-applications@plymouth.ac.uk” and follow the instructions to set a password.
- Login : <https://jupyter.lovelace.plymouth.ac.uk>
- Choose the workload profile: **General Purpose**
- Choose the resource large: **Medium (16 CPU cores, 90 GiB Memory)**
- **Open:** the notebook in ./notebooks/visualise_data.ipynb
- **Today** : we ask you to book a medium node. This is not recommended in general for debugging/development.

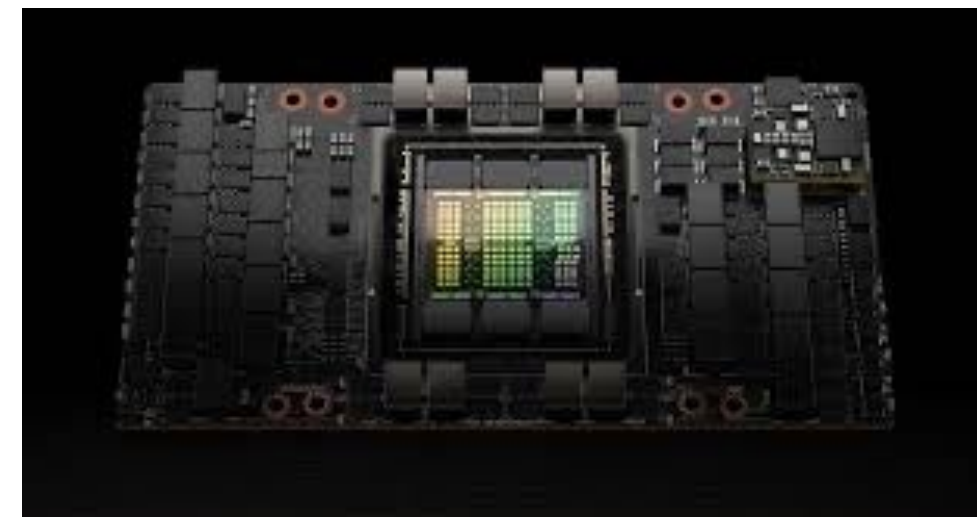
Our pipeline/roadmap

- Perform a simple preprocessing (windowing)
- Benchmark the preprocessing
- Discuss a simple MultiLayer Perceptron model
- Identify bottlenecks [more benchmarking]
- Scaling to multiple compute nodes

Note: **In a real project most of the development is NOT done on the HPC system.** The development and validation of all the tools is done on your own machine (on a subset of the data if needed). Only then the code is run on the HPC system.

Using GPUs

What is a GPU-accelerated mode ?



A GPU node is a CPU server with one or more GPU cards attached.

- The CPU is the orchestrator: runs your Python, schedules work, manages I/O
- The GPU is the accelerator: does the heavy numerical work the CPU dispatches to it
- They communicate over a high-bandwidth bus (PCIe, or NVLink between GPUs)

Our hardware:

- **H100 node:** 64 CPU cores + 4 H100 GPUs, NVLink-connected (~900 GB/s between GPUs)
- **L40s node:** 64 CPU cores + 8 L40s GPUs, PCIe-only GPU card optimised for massive parallelism

Key consequence

- the GPU can only do work fast if the CPU keeps it fed. Data has to physically travel from CPU memory to GPU memory before computation. You'll see in the benchmarks that *this transfer is often the bottleneck, not the compute.*

What is inside a GPU?

Top-level structure of an H100:

- **32 Streaming Multiprocessors (SMs)** — the GPU's parallel compute engines
- **80 GB HBM3 memory** — much faster than CPU RAM (~3 TB/s vs ~50 GB/s)
- Memory and SMs connected by a wide bus designed for parallelism

Each SM contains two different types of compute units:

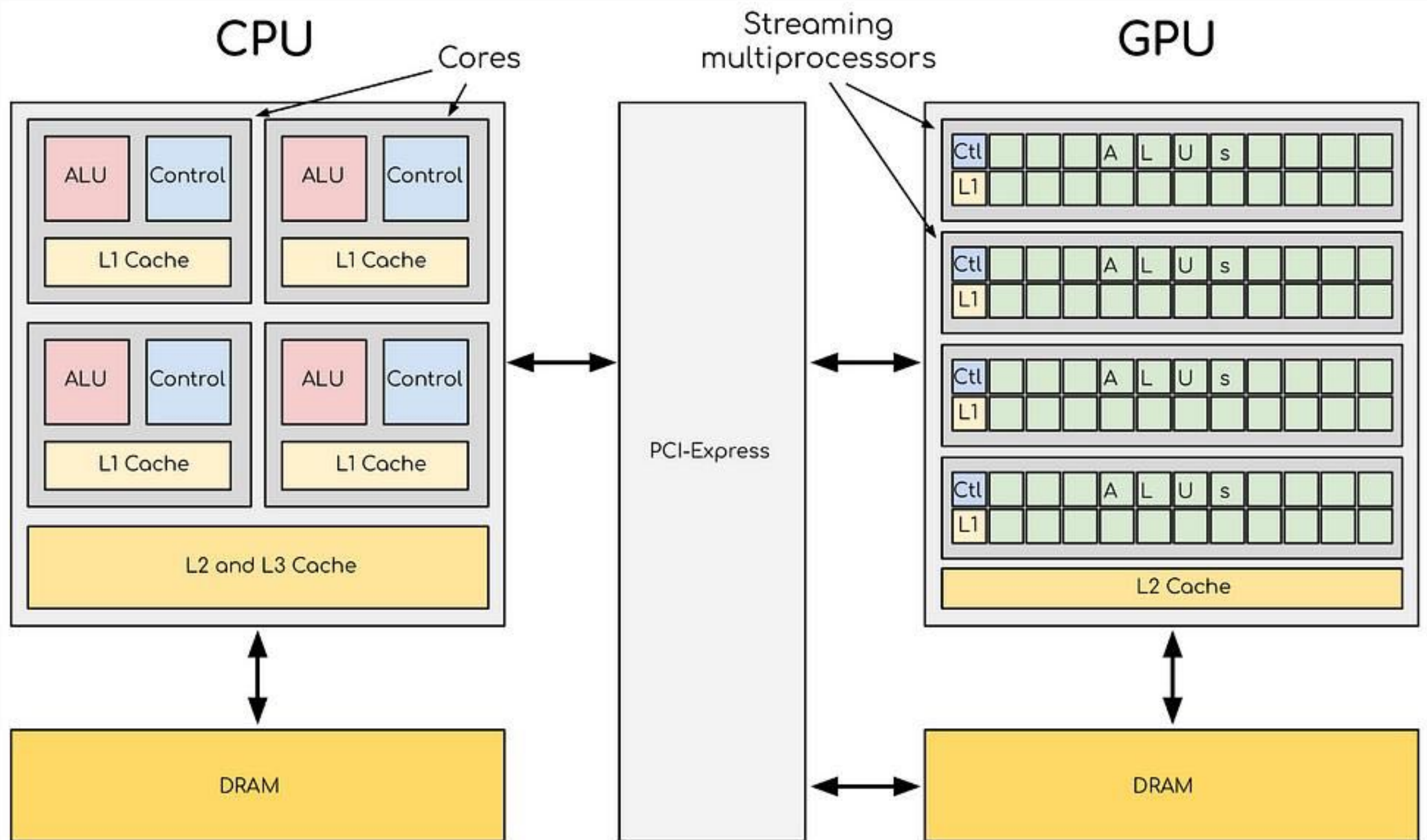
- **CUDA cores** (16,896 total on H100)
 - General-purpose arithmetic, one floating-point op per cycle each
 - Handle any precision: fp32, fp64, fp16, bf16
 - Think of them as many tiny, simple cores running in lockstep
- **Tensor cores** (528 total on H100) — separate hardware from CUDA cores
 - Specialised matrix-multiply units, one small matmul per cycle each
 - Only support reduced precision:
 - Peak throughput: ~989 TFLOPs (bf16) vs ~67 TFLOPs (fp32 on CUDA cores) — **~15× more**

H100 SXM	
FP64	34 teraFLOPS
FP64 Tensor Core	67 teraFLOPS
FP32	67 teraFLOPS
TF32 Tensor Core*	989 teraFLOPS
BFLOAT16 Tensor Core*	1,979 teraFLOPS

Key consequence

- Most neural network compute is matrix multiplication. Using Tensor cores instead of CUDA cores is where most of the speedup in modern AI training comes from.
- See for “data sheet” in <https://www.nvidia.com/en-us/data-center/h100/>

What is GPU accelerated node



How PyTorch uses all this ?

Good news : You don't program CUDA or Tensor cores directly — PyTorch hides them behind familiar operations.

- Example:
 - When you write `nn.Linear(2048, 2048)` [create a a fully-connected (dense) feedforward layer that takes 2048 input features and outputs 2048 features.]
1. PyTorch calls a pre-built matmul kernel (from cuBLAS, NVIDIA's library)
 2. The kernel inspects the input tensor's dtype
 3. **fp32 input** → CUDA cores (slow but precise)
 4. **fp32 input + TF32 enabled** → Tensor cores using TF32 (faster, slightly less precise)
 5. **bf16 / fp16 input** → Tensor cores directly (fastest)

What you control as a PyTorch user:

- `torch.backends.cuda.matmul.allow_tf32 = True` → enables TF32 globally
- `with torch.autocast(dtype=torch.bfloat16):` → forces bf16 inside the block

How a floating-point number is stored

A computer stores a number like $+3.14 \times 10^2$ in three parts:

- Sign: positive or negative (1 bit)
- Exponent: how big or small the number is (the " $\times 10^N$ " part)
- Mantissa: the digits of precision (the "3.14")

More exponent bits → bigger range (can represent very small and very large numbers)

More mantissa bits → more precision (more digits after the decimal point)

`fp32 : 32 bits (8 exp, 23 mantissa)`

`tf32 : 19 bits (8 exp, 10 mantissa) (NVIDIA chose to call "tf32" because tensors still appear to be fp32 from your code's perspective)`

`Bf16 : 16 bits (8 exp, 7 mantissa) (safer than fp16)`

`Fp16 : 16 bits (5 exp, 10 mantissa)`

For training neural networks, **range matters more than precision** (gradients can be very small numbers). That's why `bf16` is the modern default and `fp16` needs special handling (`GradScaler`) to avoid gradient underflow.

First gpu run

- Go in `./ecg_ai/slurm`
- The following command will submit a job to the H100 queue [edit the file to submit to l40s instead]

```
sbatch gpu_burn.slurm
```

- Then type to go to the gpu node

```
ssh gnode01 [or gnode02 for the L40s]
```

- To visualise how busy is the gpu type :
`nvidia-smi`

Note: there are only 12 GPUs in total

First gpu run

```
(base) [vdrach@gnode01 ~]$ nvidia-smi
Mon Jun 22 16:37:06 2026

+-----+
| NVIDIA-SMI 550.90.07                  Driver Version: 550.90.07          CUDA Version: 12.4        |
+-----+-----+-----+-----+-----+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf              Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                                           | MIG M.         |
+-----+-----+-----+-----+-----+-----+
|  0    NVIDIA H100 80GB HBM3              Off | 00000000:06:00.0 Off |   0          Default |
| N/A   52C    P0              684W / 700W |  905MiB / 81559MiB | 100%        Disabled |
+-----+-----+-----+-----+-----+-----+

+-----+
| Processes: |
| GPU   GI    CI          PID    Type    Process name                  GPU Memory |
|          ID    ID                                   |          Usage |
+-----+-----+-----+-----+-----+-----+
|  0     N/A   N/A      1654787      C      python                        894MiB |
+-----+-----+-----+-----+-----+-----+
```

Slurm script

- Type

less ./slurm/gpu_burn.slurm

```
#!/bin/bash -l
#SBATCH -J AI
#SBATCH -N 1
#SBATCH -n 4
#SBATCH -t 12:00:00
#SBATCH -p gpu_h100
#SBATCH --mincpus=4
#SBATCH --gres=gpu:1
#####SBATCH -o DNA_out_1000000000_N1

source /users/${USER}/ecg_ai/modules_for_ecg_ai
. /users/${USER}/.jupyter_virtualenvs/HPC_pytorch_AI/bin/activate
python ../scripts/gpu_burn.py --seconds 120
```

(jobname)
(Number of gpu node)
(Number of cpu cores)
(Wall time)
(Partition)
(Number of GPUs: 1 for now)
Load modules
Activate virtual environment
Run code !

Slurm cheat sheet

- sbatch: submit a job.
- squeue : to view the queue, in particular the jobID
- squeue -u vdrach: view only my jobs
- scancel jobid : to cancel job with “jobid”

Preprocessing

Preprocess [./scripts/preprocess.py]

Goal:

- read the raw data
- define non overlapping widows of length L (10 s by default).
- Create an array **X** of shape (n_windows, 1280, n_channel=2).
- Create the label vector **y** of shape (n_windows,) : 1 is AF, 0 is normal
- Save the data (one fsile per participant, in a compressed format).

Comments:

- Pretty basic preprocessing.
- More involved preprocessing could involve feature engineering [potentially with Dask]
- Typically not a GPU task

Note: **Code developed and tested on my laptop on a tiny subset o 3 patients.**

Preprocessing

Comments:

- There is a slurm script in ./slurm
- Submit it. It takes ~15 min
- Is it a parallel code ? Is it a good idea to book 8 cores ?
- Is the current code efficient on 8 cores ? On 1 ?

◦

```
#!/bin/bash -l
#SBATCH -J preprocess
#SBATCH -N 1
#SBATCH -n 8
#SBATCH -t 01:00:00
#SBATCH -p cpu_shared
#####SBATCH -o DNA_out_1000000000_N1

source /users/${USER}/ecg_ai/modules_for_ecg_ai
. /users/${USER}/.jupyter_virtualenvs/HPC_pytorch_AI/bin/activate

python ../scripts/preprocess.py --data-dir /users/vdrach/ecg_ai/data/raw/ltafdb --output-dir /users/vdrach/ecg_ai/data/preprocessed &
```

About efficiency

- Now we could ask ourselves is the single core using by serial code used efficiently ?
- Simple way to “profile” or tool, would be to ssh to the node doing the preprocessing (ssh cnodeXY , you can find XY by typing `squeue`)and then type:

`top`

○

```
top - 21:56:21 up 53 days,  3:48,  2 users,  load average: 21.15, 20.64, 20.52
Tasks: 889 total,  22 running, 867 sleeping,   0 stopped,   0 zombie
%Cpu(s): 32.7 us,  0.1 sy,  0.0 ni, 67.0 id,  0.0 wa,  0.2 hi,  0.0 si,  0.0 st
MiB Mem : 386149.4 total, 361692.4 free,  20599.3 used,  3857.7 buff/cache
MiB Swap:  4096.0 total,  4096.0 free,    0.0 used. 363062.9 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
355616	cjohann+	20	0	13.8g	506216	18528	R	99.7	0.1	430:40.90	mitgcmuv
355635	cjohann+	20	0	14.5g	503948	22024	R	99.7	0.1	430:39.71	mitgcmuv
355618	cjohann+	20	0	1611184	502056	18332	R	99.3	0.1	430:40.08	mitgcmuv
355620	cjohann+	20	0	4946920	499944	18392	R	99.3	0.1	430:35.86	mitgcmuv
355621	cjohann+	20	0	13.6g	504732	22864	R	99.3	0.1	430:32.56	mitgcmuv
355623	cjohann+	20	0	5976844	504736	22816	R	99.3	0.1	430:41.51	mitgcmuv
355625	cjohann+	20	0	5431068	510828	21728	R	99.3	0.1	430:40.87	mitgcmuv
355627	cjohann+	20	0	14.3g	504976	21780	R	99.3	0.1	430:37.51	mitgcmuv
355628	cjohann+	20	0	960220	507404	21696	R	99.3	0.1	430:36.66	mitgcmuv
355631	cjohann+	20	0	9060612	503804	21768	R	99.3	0.1	430:40.67	mitgcmuv
355632	cjohann+	20	0	9826628	503796	22020	R	99.3	0.1	430:31.68	mitgcmuv
355633	cjohann+	20	0	7105492	505632	22048	R	99.3	0.1	430:35.38	mitgcmuv
355634	cjohann+	20	0	6512552	507712	21872	R	99.3	0.1	430:39.03	mitgcmuv
355617	cjohann+	20	0	14.8g	504660	22752	R	99.0	0.1	430:39.05	mitgcmuv
355619	cjohann+	20	0	13.5g	504776	22836	R	99.0	0.1	430:32.23	mitgcmuv
355622	cjohann+	20	0	1132848	500308	18392	R	99.0	0.1	430:31.00	mitgcmuv
355624	cjohann+	20	0	6151516	507220	21764	R	99.0	0.1	430:39.90	mitgcmuv
355626	cjohann+	20	0	1171160	507512	21732	R	99.0	0.1	430:33.31	mitgcmuv
355629	cjohann+	20	0	2667100	503724	21720	R	99.0	0.1	430:38.64	mitgcmuv
355630	cjohann+	20	0	11759	505732	22020	R	99.0	0.1	430:34.02	mitgcmuv
580725	vdrach	20	0	8747544	188208	19792	R	99.0	0.0	0:12.33	python

About efficiency (2)

- But we are nice so we provide you a tool in ./scripts : resource_monitor.py [that we will provide centrally and document soon]
- In this example it runs “top” every second and record the information and log them in a preprocessed_cpu_shared.csv
- **Have a look a confirm that the “proc_cpu_pct” is close to 100% at all times!**

```
TRAIN_PID=$!  
python ../scripts/resource_monitor.py --interval 1 --out preprocess_${SLURM_JOB_PARTITION}.csv --pid ${TRAIN_PID} &  
MONITOR_PID=$!  
wait $TRAIN_PID  
kill $MONITOR_PID 2>/dev/null # then stop the monitor cleanly  
wait $MONITOR_PID 2>/dev/null # let it flush/exit
```

Comments/Exercise

- This is an embarrassingly parallel task : processing patient 1 and 2 could occur at the same time on 2 distinct core having access to the shared memory.
- Left as an exercise. You can use `ProcessPoolExecutor` in `concurrent.futures`
- We expect : run on 8 cores should be 8 times faster !
 - Untested solution is provided [`preprocessed_parallel.py`]

Training of a simple model

The model — a simple MLP baseline

Input: (batch, 1280, 2) ← 10-second ECG window, 2 leads

↓ Flatten

(batch, 2560)

↓ Linear(2560 → 128)

(batch, 128)

↓ ReLU

↓ Linear(128 → 2)

Output: (batch, 2) ← logits for {non-AF, AF}

Parameter count: ~328K

To convert the output into a probability : `torch.softmax(logits, dim=1)`

The dataset class

- Three variants in the code, varying how much work happens when:

```
class ECGDataset:                # "slow"  
    # Reads from disk on every __getitem__ call  
    # -> I/O cost paid per sample, per epoch  
  
class CachedECGDataset:          # "medium"  
    # Reads all files once at __init__, keeps in CPU RAM  
    # -> I/O cost paid once at startup  
  
class FastCachedECGDataset:      # "fast"  
    # Reads all files once, concatenates into one big tensor (CPU RAM)  
    # -> I/O paid once, no per-sample Python overhead
```

Two practical points worth flagging:

1. The "fast" version requires the whole dataset to fit in CPU RAM (~7 GB here, fine)
2. `__getitem__` in the fast version is just `return self.X[idx], self.y[idx]` — pure tensor indexing, essentially free

The dataloader

- PyTorch's `DataLoader` is a thin wrapper around your `Dataset` that handles the bookkeeping of training: grouping samples into batches, shuffling, optionally using subprocess workers to prefetch data while the GPU is busy. Basically, the `DataLoader`'s job is to take individual samples from your `Dataset` and group them into batches.

```
from torch.utils.data import DataLoader
loader = DataLoader(
    dataset,
    batch_size=128,
    shuffle=True,
    num_workers=0,
)
for x, y in loader:
    # x: tensor of shape (128, 1280, 2)
    # y: tensor of shape (128,)
```

- Note that `num_workers` How many subprocess workers prepare batches in parallel. 0 means the main process does all the work itself

Internally, on each iteration it:

1. Picks the next 128 indices (in shuffled order, since `shuffle=True`)
2. Calls `dataset.__getitem__(idx)` for each → gets 128 individual samples
3. **Stacks them** into one batched tensor (this step is called "collation")
4. Returns the batched `(x, y)` to your training loop

Some (cpu) benchmarks

- On my Mac for 3 patients to run `benchmark_dataset.py`
 - 3068.22 s (slow class)
 - 0.1 s CachedECGDataset (medium)
- Varying the number of workers (medium class) :
 - 0 workers : 0.1 s
 - 2 workers : 12 s
 - 4 workers : 24 s
- That's the **opposite** of what people usually expect from adding workers
- Takeaway: Experiment and measure timings.

MLP training : CPU vs GPU

- **To run on GPU (cuda here) : appreciate the simplicity**

```
model = MLP().to("cuda")
```

- **Benchmark runs 10 epochs with batch_size=256 and dataclass fast**
- **Full cpu node (64 cores) : ~ 300 sec per epoch**
- **H100 : ~6.5 sec per epoch**

Are we done ?!

MLP training : Are we using the H100 efficiently ?

- **Suggestions ?**

MLP training : Are we using the H100 efficiently ?

When training feels slow, the first question is always: *where is the time going?* Three tools :

- Print timings (option `—profile`)
- Use the `ressource_monitor.py`
- `torch.profiler`

Let's start with the first 2. Submit `1_submit_MLP.slurm`

MLP training : Stage-by-stage timing

I find ~ 7 sec/epoch

Loading	: 32.9%
Transfer	: 4.9%
Forward	: 18.2%
Backward	: 24.7%
Optimizer	: 19.3%

Important subtlety: GPU operations are asynchronous. When you call `model(x)`, PyTorch queues the work and returns immediately — the GPU hasn't actually finished yet. To time it correctly you need a synchronisation barrier.

The cost of synchronisation isn't free

Warning : time measured isn't just compute — it's also a sequence of kernel launches, and the launches have fixed overhead. At a small batch size, each kernel finishes in less time than it took the CPU to dispatch it — so total wall time is dominated by the dispatching, not the computing.

Before interpreting too much let's look at the output using the `resource_monitoring` script

MLP training : Resource monitoring

The resource monitoring should have created : `train_MLP_gpu_h100.csv`

Let's visualise the result.

Open the notebook.

node CPU : The CPU usage of the **entire compute node (100% means all 64 cores are fully bus)**

process CPU : what is my specific job doing on the CPU

process RSS : **Resident Set Size** — the amount of physical RAM your process currently has allocated and in use, in GB.

We conclude that GPU is on 15% of the time and that the memory is underused.

MLP training : profiler

Now run 2_submit_MLP.slurm

```
source modules_for_ecg_ai  
. /users/vdrach/.jupyter_virtualenvs/HPC_pytorch_AI/bin/activate
```

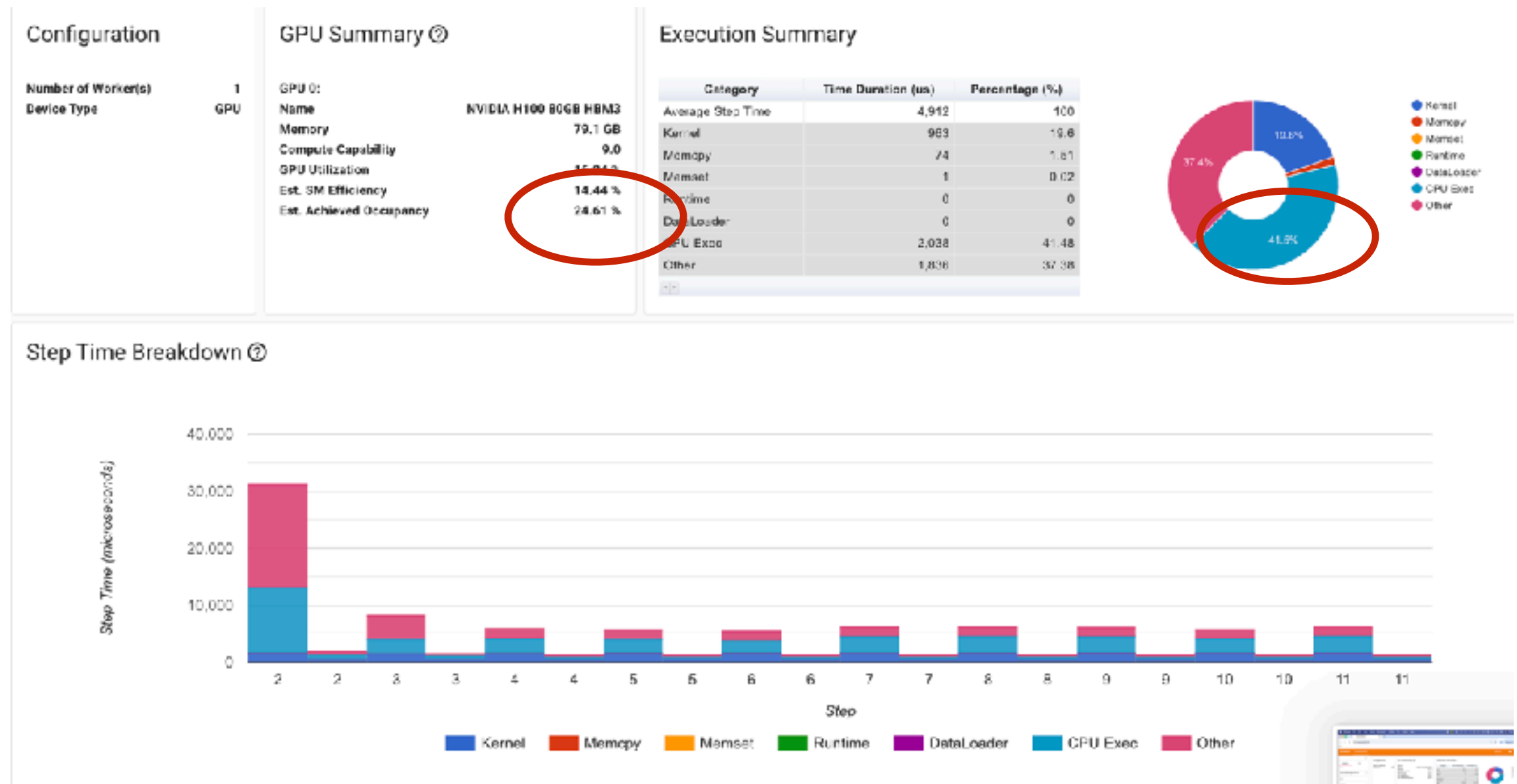
```
tensorboard --logdir torch_profiler_logs_1691415 --port 6025 --  
bind_all
```

```
ssh -L 6025:localhost:6025 username@lovelace.plymouth.ac.uk
```

Then open in your browser

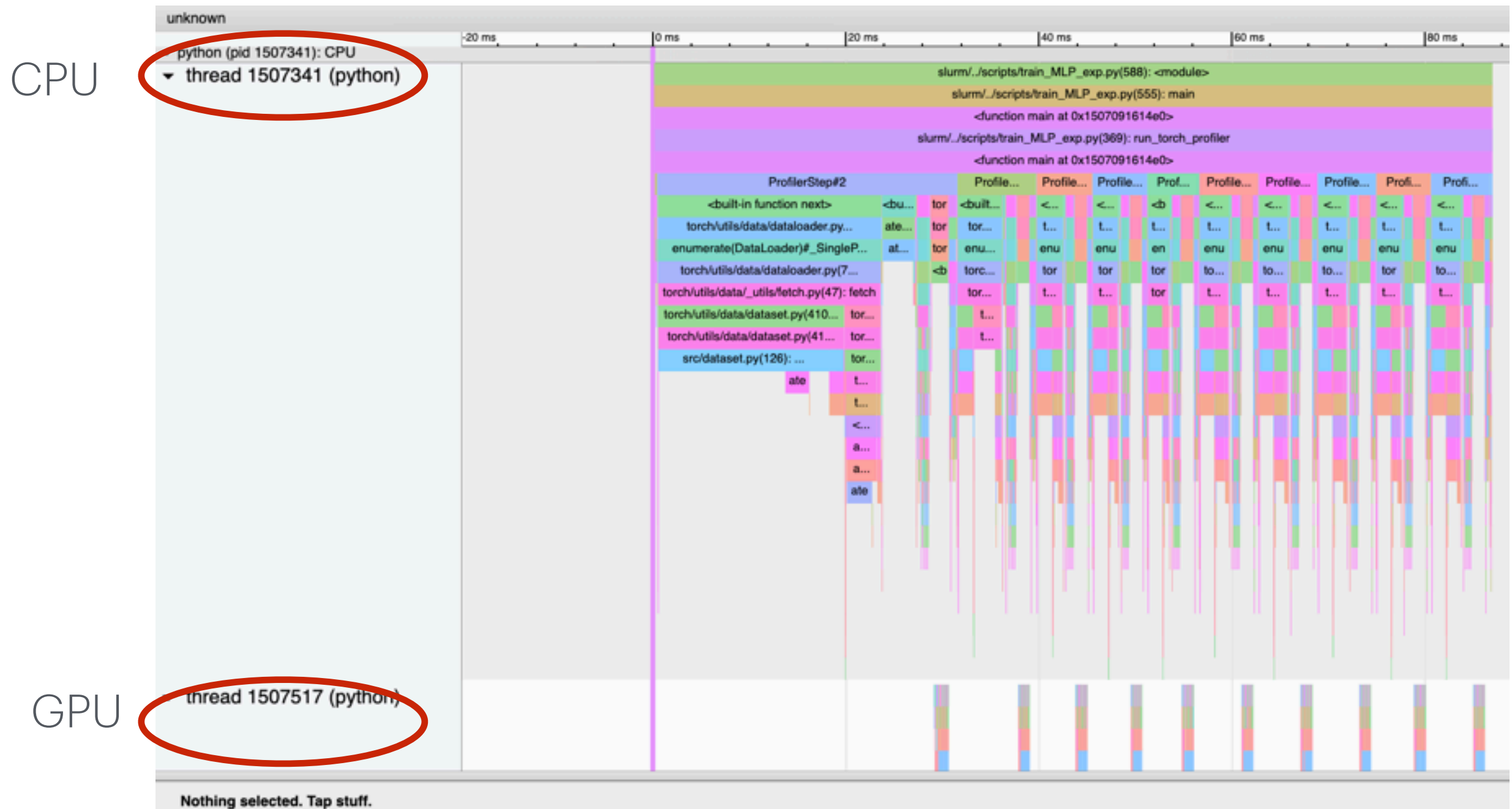
```
http://localhost:6025
```

MLP training : profiler



- Also written : “GPU 0 has low utilization. You could try to increase batch size to improve. Not Increasing batch size may affect the speed and stability of model convergence.”

MLP training : profiler



- Selecting the trace option in the menu.
-

MLP training : conclusion

GPU code looks fast but the H100 is underutilised.

The issue: not enough work per GPU operation — small model + small batch.

Sequence of kernel launches is probably slowing us down: GPU spends most of its time waiting for the CPU to dispatch the next kernel — fixed overhead per launch dominates when the actual compute per launch is tiny.

Exploring options

Optimisations :

- **Give the GPU more work per launch:**
 - Increase batch size using `—batch_size 8096`
 - Change the model using `—model big` (or `tiny` or `huge`)
- Use the faster hardware (Tensor cores):
 - enable TF32 for fp32 matmul
 - Enable `mixed_precision`
 - Choose the target type if `mixed_precision` is on using `--amp_dtype` (`bf16` or `fp16`)
- **Reduce data-movement overhead:**
 - Use the `—pin_memory` option to enable faster CPU→GPU transfers via pinned (page-locked) host memory
 - Try to force the data on the gpu to minimise transfer to the gpu using `—data_on_gpu`

The rest of the workshop is about trying these systematically and measuring what each one does — not picking and choosing based on intuition.

Snakemake: Reproducible & Scalable Workflows

What is Snakemake?

- A workflow management system for computational experiments and benchmarks
- Automatically determines which steps need to run
- Supports local execution, clusters, and cloud environments
- Helps make analyses **reproducible**, **organized**, and **scalable**

See for instance : Snakefile and `benchmark_MLP_snakemake.slurm`

(Alternative without snakemake `benchmark_MLP.slurm`)

```
rule benchmark:
    input:  "data/input.txt"
    output: "results/output.txt"
    shell:
        "my_program {input} > {output}"
```



Discussion: results

Results can be viewed in notebook

- Discuss the results

Distributed training

Several parallelisation strategies

Distributed training: scaling beyond one GPU

- Parallelisation is **not** guaranteed to be efficient — speedup is bounded by communication overhead
- **Don't parallelise inefficient code.** If single-GPU is under-utilised, multi-GPU will be even worse (more idle GPUs, plus communication on top)
- PyTorch offers several strategies: data parallelism, model parallelism, pipeline parallelism, tensor parallelism, FSDP
- **We use the simplest: data parallelism** — each GPU has a copy of the model, processes a different shard of the data, gradients are averaged across GPUs after each backward pass (via `DistributedDataParallel` + `torchrun`)
- Two architectures to compare scaling behavior:
 - 1D ResNet (compute-heavy convolutions)
 - CNN + Transformer dual-branch (compute + communication mix)

Data parallelism — what actually changes in the code

Comparison single vs multi-gpu code:

single gpu code

```
model = ResNet1d(...).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(epochs):
    for x, y in batches:
        logits = model(x)
        loss = criterion(logits, y)
        loss.backward()
        optimizer.step()
```

multi gpu code

```
dist.init_process_group(backend="nccl")           # +1 line
torch.cuda.set_device(local_rank)                # +1 line

model = ResNet1d(...).to(device)
model = DDP(model, device_ids=[local_rank])      # +1 line (wrap)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(epochs):
    for x, y in batches_for_this_rank:           # data sharding
        logits = model(x)
        loss = criterion(logits, y)
        loss.backward()
        optimizer.step()
```

Discussion

- Submit `benchmark_scaling_snakemake.slurm`
- Interpret or better plot the timing per epoch vs the number of GPU card. Does the code scale well ?
- Compare H100 with L40s?

Conclusion

Takeaways

- **Profile before optimising.**
- **Each optimisation targets a specific bottleneck.**
- **DDP is easy to enable, harder to make efficient.**

A 10× speedup does not just save time:

- It may allow you to run 10× more simulations.
 - It may allow you to increase the problem size dramatically.
 - It may make previously infeasible computations possible.
-
- Experiment and automatise!